

AWS ELB

HA & Scalability

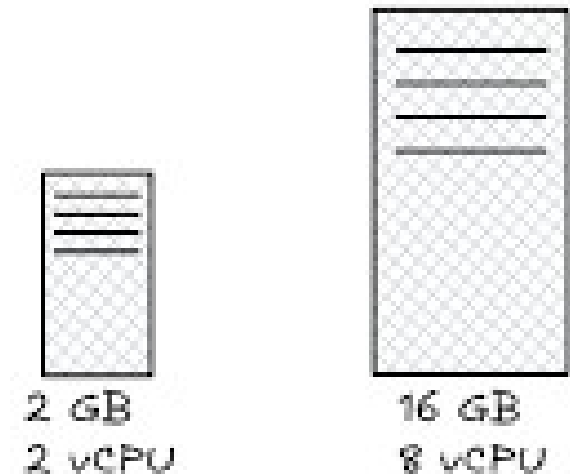
Scalability "Handling the Crowd"

- Scalability is the ability of your architecture to handle a growing (or shrinking) amount of work by adding or removing resources. It's all about **capacity**.
- Example: If your website suddenly gets featured on the news and traffic spikes by 1000%, a scalable system adjusts to handle the load without crashing.
- Two ways to scale:
 - Vertical Scaling (Scaling Up/Down)
 - Horizontal Scaling (Scaling Out/In)

Vertical Scaling (Scaling Up/Down)

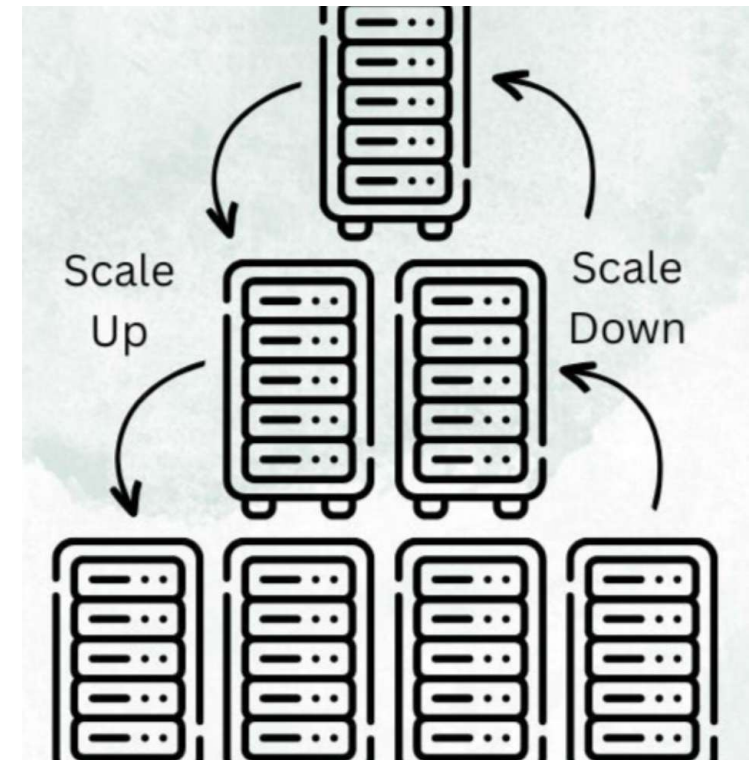
- Means making your existing server bigger.
- In EC2, this means changing your instance type from a small one (e.g., t3.micro with 1 vCPU) to a massive one (e.g., m5.4xlarge with 16 vCPUs).
- It usually requires a brief moment of downtime to resize the server.
- Catch: Every server has a hard physical limit on how big it can get.

Vertical Scaling



Horizontal Scaling (Scaling Out/In)

- Means adding *more* servers of the same size.
- In EC2, this means, instead of one giant EC2 instance, you run five small ones.
- **In EC2**, This is managed by **Auto Scaling Groups (ASG)**. You set rules (e.g., "*If CPU usage goes above 70%, launch two more EC2 instances*"). When traffic drops, it automatically terminates the unneeded instances to save you money.
- Catch: You cannot achieve Horizontal scaling without Load Balancers.



High Availability (HA): "Never Going Down"

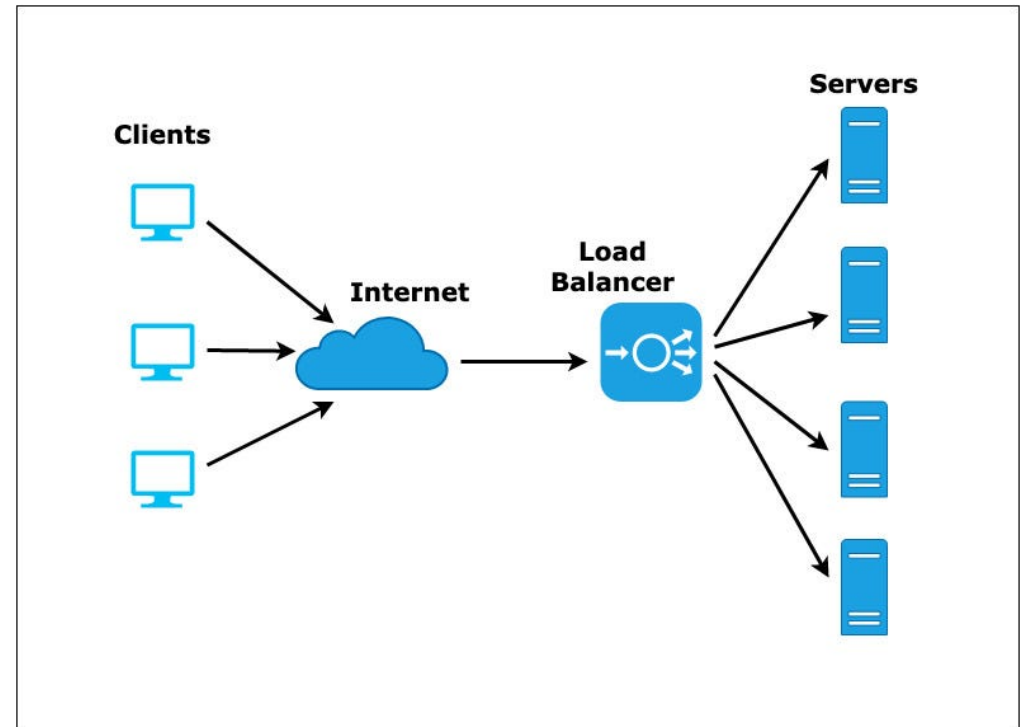
- Means your application is designed to tolerate failures and remain operational with minimal to no downtime. It's all about **redundancy**.
- Example: You run your application on a single EC2 instance and that instance fails (or the underlying AWS hardware suffers an outage), your website goes down.
- To achieve HA, you distribute your infrastructure:
 - **Multi-AZ Deployment:** You run identical EC2 instances in different **Availability Zones** (separate, isolated data centers within a region).
 - **Load Balancer (ALB):** You place an Elastic Load Balancer in front of those instances. If an instance or an entire data center goes offline, the load balancer automatically reroutes traffic to the healthy instances in the other zone.

Load balancer

Load Balancer

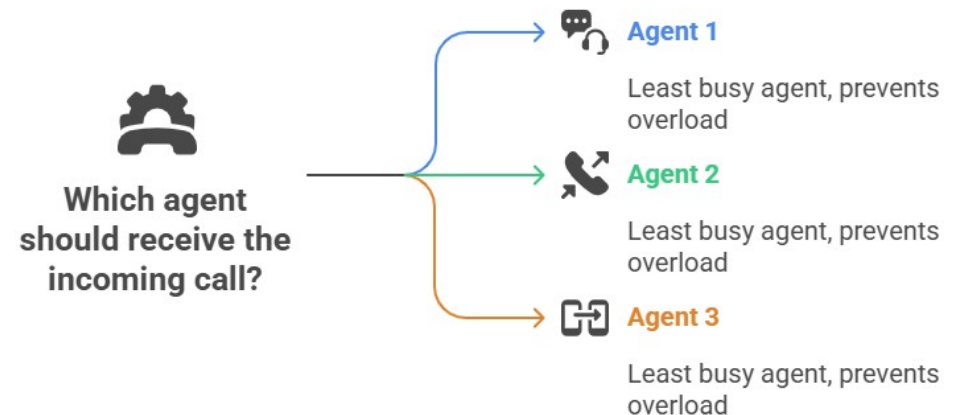
- At its simplest, a **Load Balancer** is a device or software that acts as a **traffic cop** for your servers.
- Load Balancing is the process of distributing incoming traffic evenly across multiple servers (e.g., EC2 instances) to ensure:
 - High availability
 - Fault tolerance
 - Better performance
 - Scalability

In AWS, this is done using the Elastic Load Balancer (ELB) service.



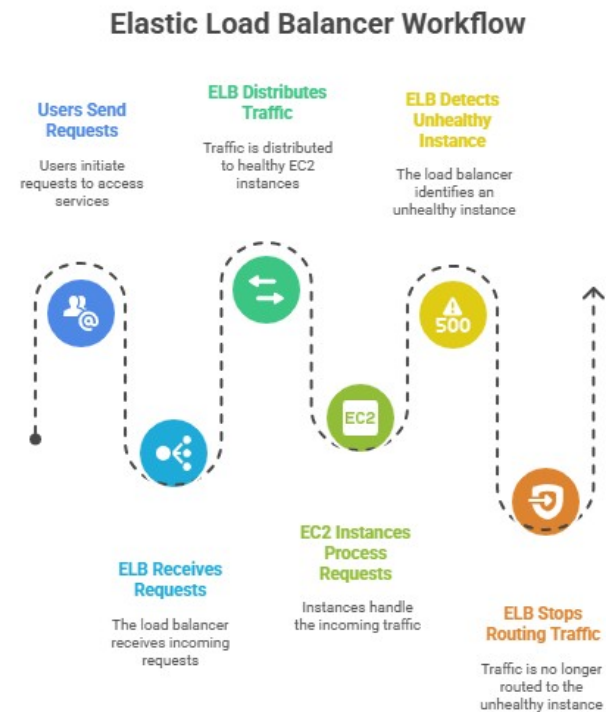
LB Analogy

- Imagine a call center with **three agents (EC2 instances)** and one **receptionist (Load Balancer)**. The receptionist sends each incoming call to the **least busy agent** — so no single agent gets overwhelmed.



How It Works

- Users send requests (e.g., website hits, API calls).
- The **Elastic Load Balancer** receives the requests.
- It **distributes traffic** to multiple healthy **EC2 instances** behind it.
- If one instance becomes unhealthy, ELB automatically **stops routing traffic** to it.

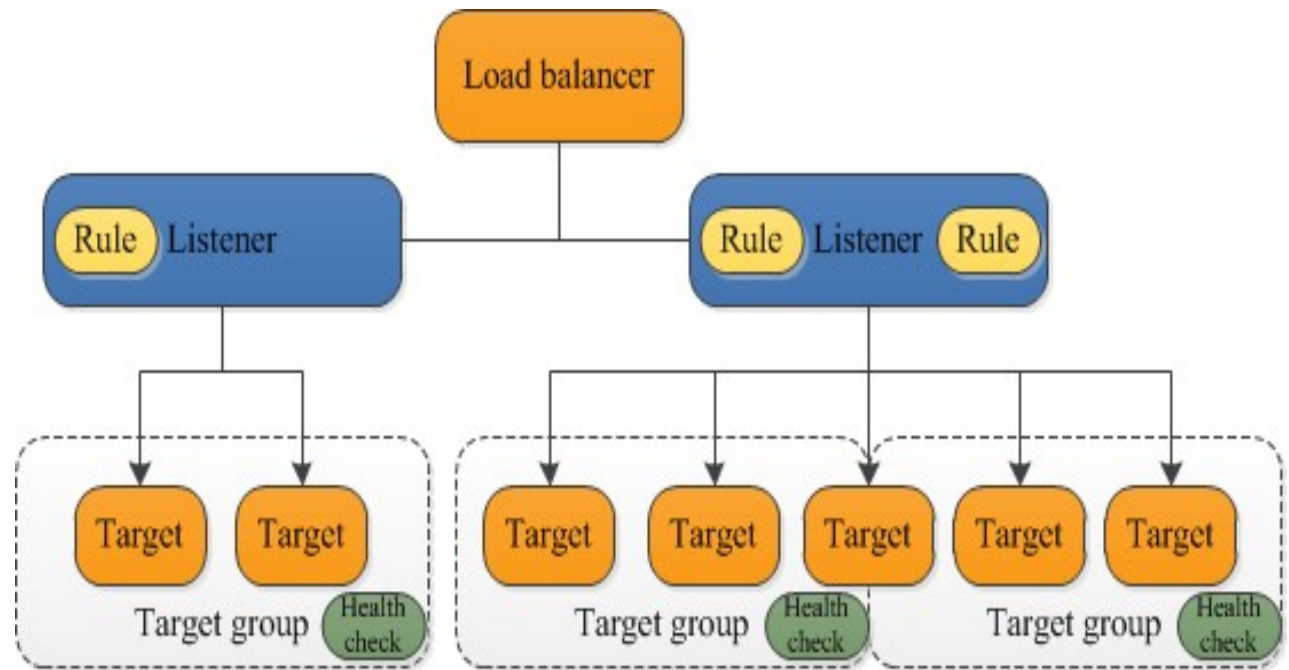


Why Use a Load Balancer?

- **Distribute Workload Evenly : Spread load across multiple downstream instances**
 - Prevents overloading one server and ensures better response time.
 - ✓ Example: 4 EC2 instances behind one ALB handle 4× more requests efficiently.
- **Single Point of Access (DNS Entry) :** The ELB provides a **single DNS name** (e.g., myapp-123.elb.amazonaws.com)
 - Users always connect through this endpoint, regardless of backend instance changes.
 - ✓ Simplifies management and failover.
- **High Availability: Multi-AZ Load Balancers** ensure **uptime even if one AZ fails**.
 - Traffic automatically shifts to healthy targets in another AZ.
- **Fault Tolerance via Health Checks :** Performs **regular health checks** on downstream EC2 instances.
 - Automatically removes unhealthy targets until they recover.
- **Security & HTTPS Offloading : SSL/TLS termination** at the Load Balancer level (HTTPS support).
 - Frees backend servers from the CPU-intensive SSL decryption workload.
 - ✓ Easier certificate management (ACM integration).
- **Session Stickiness :** Can **enforce stickiness with cookies** — routes a user's requests to the same backend instance.
 - Useful for **stateful applications** (e.g., shopping carts, user sessions).

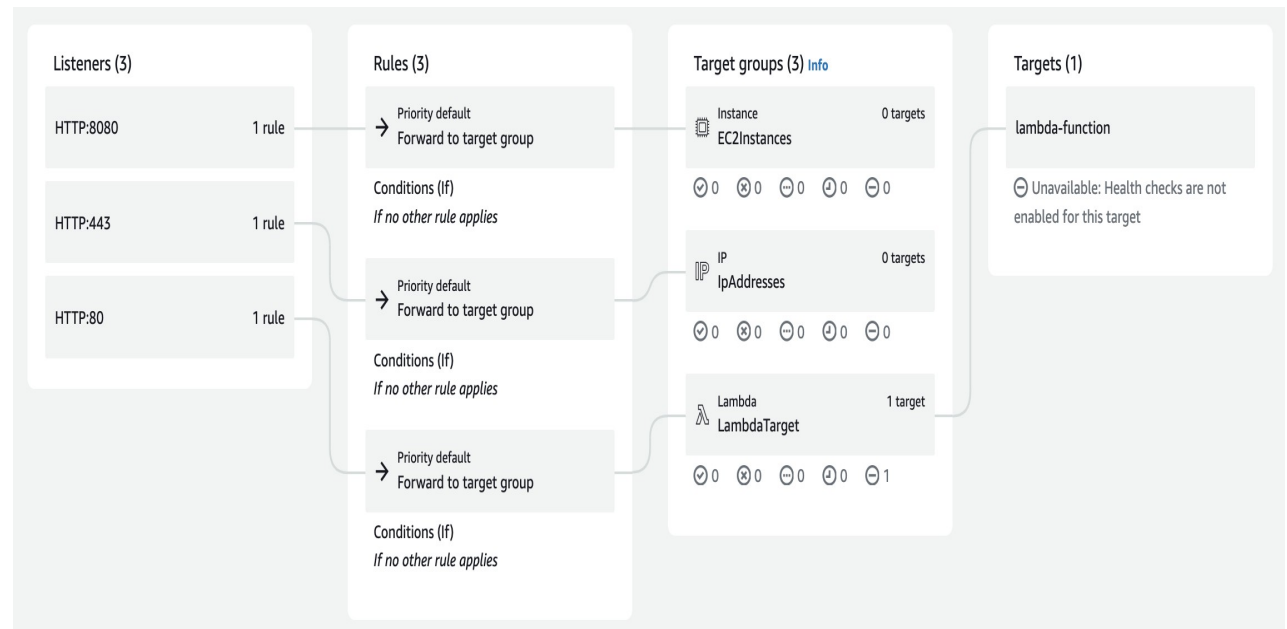
Load Balancer components

- Load balancer (The Entry Point)
 - serves as the single point of contact for clients.
 - distributes incoming application traffic across multiple targets.
 - one or more listeners can be added to the load balancer.
- Listener (The Gatekeepers)
 - checks for connection requests from clients, using the configured protocol and port
 - rules defined determine, how the load balancer routes request to its registered targets.
 - each rule consists of a priority, one or more actions, and one or more conditions.



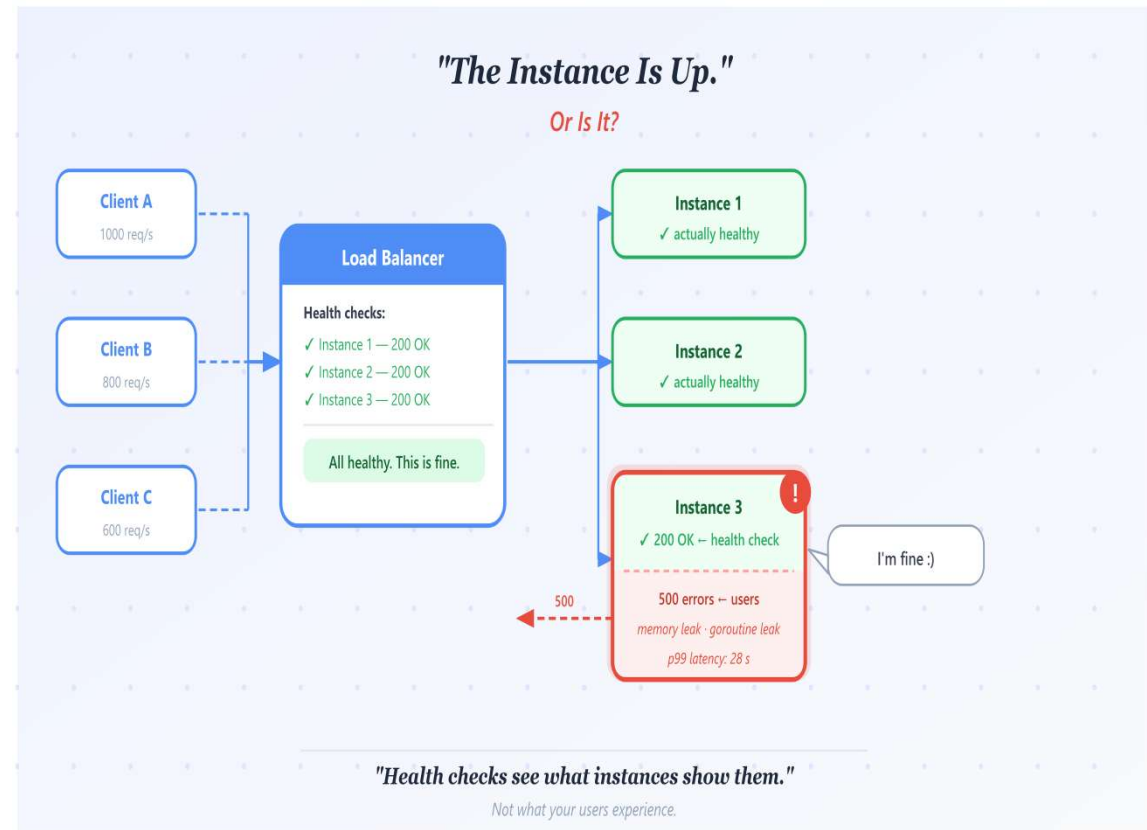
Load Balancer components

- Target group (The Destinations)
 - routes requests to one or more registered targets, such as EC2 instances, using the specified protocol and port number
 - a target can be registered with multiple target groups.
 - health checks can be configured on a per target group basis.
- Routing algorithm:
 - **Round Robin** (default) – distributes requests evenly across targets
 - **Least Outstanding Requests (LOR)** – routes to the target with fewest pending requests
 - **Weighted Random** – used by Automatic Target Weights (ATW) for anomaly mitigation



Health Checks in LB

- Health Checks are crucial because they enable the load balancer to know if the instances it is routing traffic to are actually available.
- The load balancer routinely pings a specific port and route (e.g., /health).
- If the instance does not respond with a 200 OK status, the load balancer marks the instance as unhealthy and stops routing traffic to it.



Types of AWS Load Balancers

4 Types of AWS Load Balancers

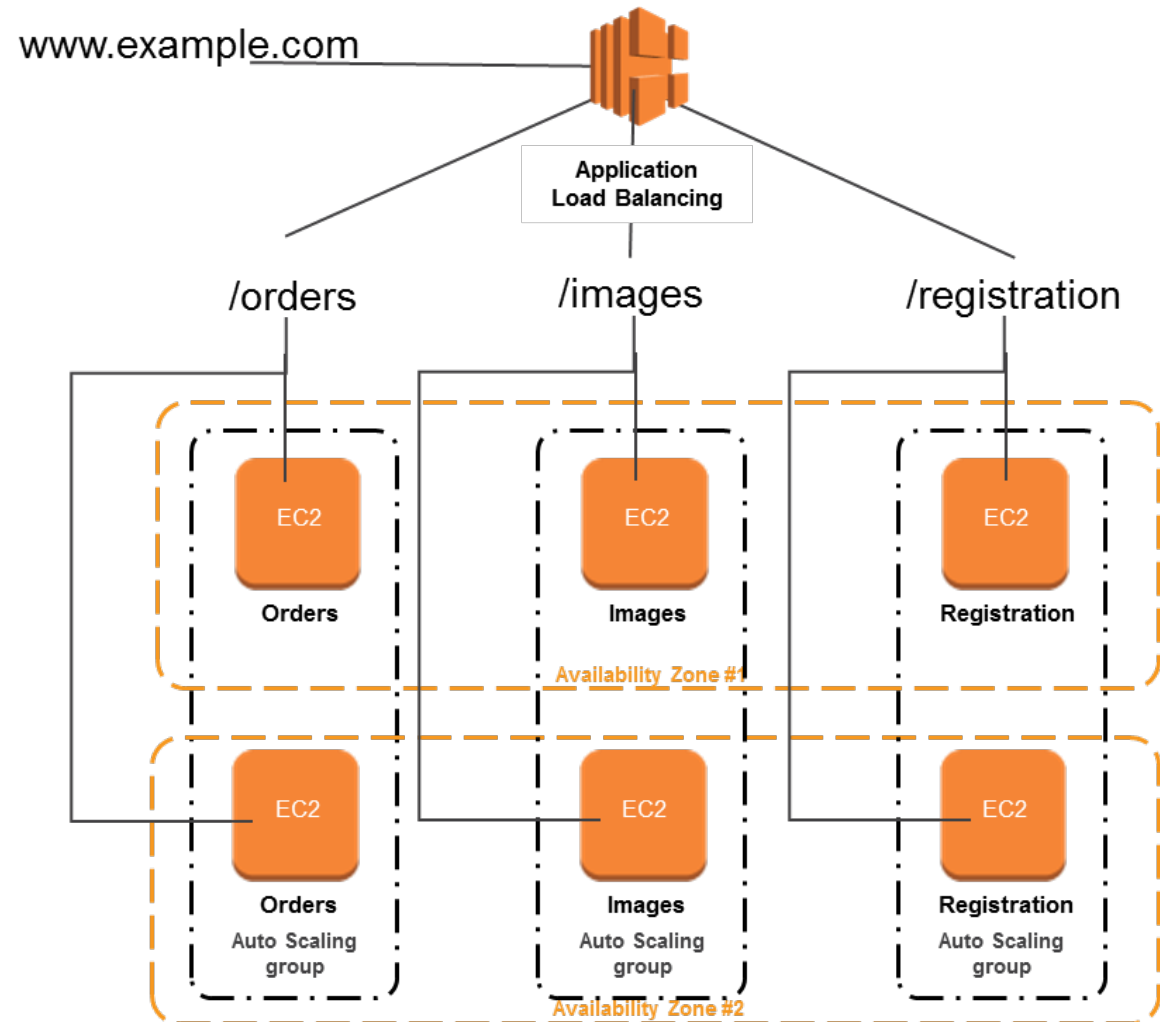
- **Application Load Balancer**
- **Network Load Balancer**
- **Gateway Load Balancer**
- **Classic Load Balancer**

Application Load Balancer (ALB)

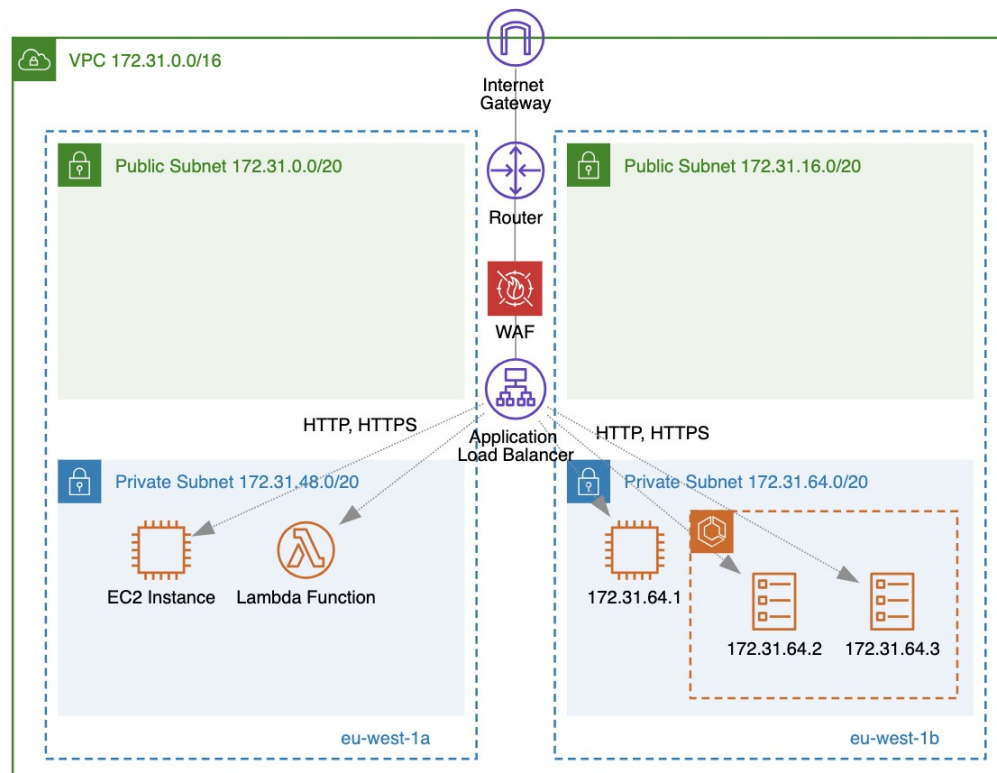
- The AWS Application Load Balancer (ALB) operates at the application layer (Layer 7) of the OSI model, making it ideal for handling HTTP and HTTPS traffic.
- ALB is designed for advanced routing and enables complex application architectures that require path-based, host-based, and query-based routing decisions.
- Supports WebSockets and Secure WebSockets natively.
- **Features of ALB**
 - **Layer 7 Routing:** ALB can inspect application-level details within the request headers to make routing decisions, allowing fine-grained control over request handling.
 - **Path- and Host-Based Routing:** ALB can direct traffic based on URL paths and hostnames, making it a perfect fit for microservices architectures.
 - **WebSocket Support:** ALB supports WebSocket connections, which are essential for real-time applications that require long-lived connections.
 - **SSL Offloading:** ALB offloads the SSL/TLS termination, reducing the computational load on backend servers and enhancing security.
 - **Sticky Sessions:** ALB supports session stickiness, allowing users to maintain their session on a single target throughout their interaction.
 - **ALB Target Group Protocols:** HTTP, HTTPS, and gRPC.

ALB Use Case

- Use Case: Application Load Balancers are best suited for microservices-based applications, API services, and any architecture that relies on Layer 7 routing features. They are also ideal for scenarios where there is a need for advanced HTTP/HTTPS management.



ALB



ALB Limitation

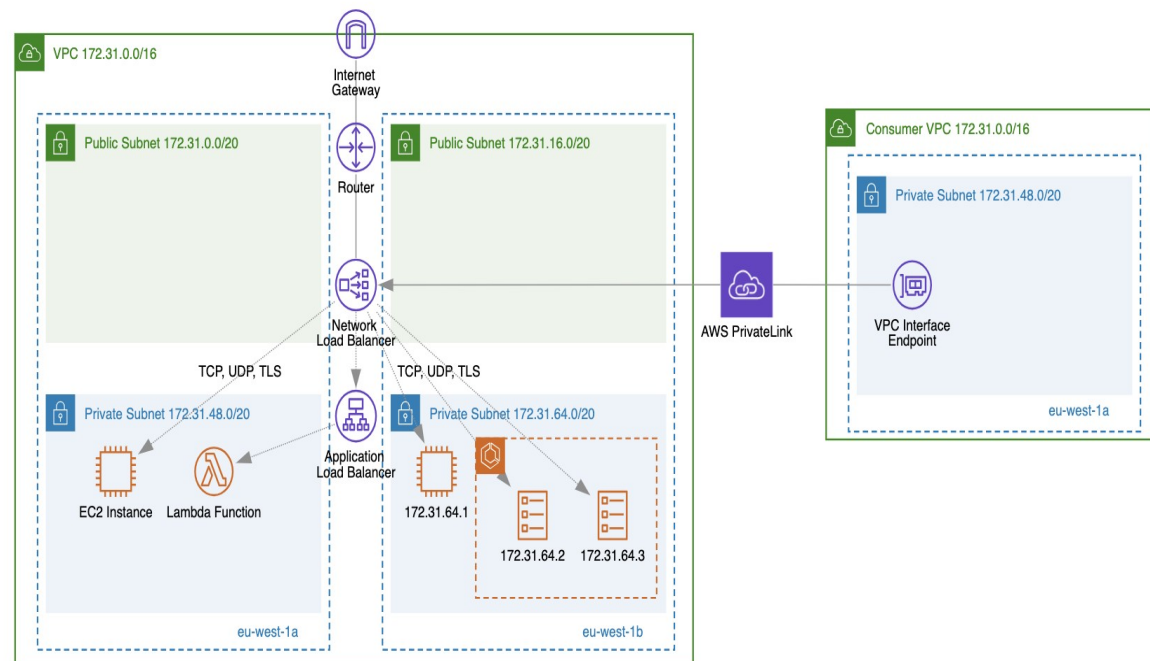
- An Application Load Balancer (ALB) **only supports Layer 7 (Application Layer) web protocols**. Specifically, it supports **HTTP, HTTPS, WebSockets, and gRPC**. Because it is locked into the web application layer, it comes with several hard architectural limitations. If your system requires capabilities beyond these protocols, an ALB cannot support them.
- **No Static or Elastic IPs:** An ALB cannot be assigned a fixed, unchanging IP address. AWS dynamically scales the ALB's underlying nodes, meaning its IP addresses change constantly. You are forced to route traffic to it using a DNS name (example-1234.amazonaws.com).

Network Load Balancer (NLB)

- The Network Load Balancer (NLB) operates at the transport layer (Layer 4) and is designed for high-performance applications that require ultra-low latency. NLB is highly effective at handling millions of requests per second, making it a preferred choice for latency-sensitive workloads.
- **Features of NLB**
 - **Layer 4 Load Balancing:** NLB uses TCP and UDP protocols, providing fast and straightforward load balancing for applications requiring minimal routing complexity.
 - **Static IP Addresses:** NLB assigns a single static IP per Availability Zone, simplifying the management of IP addresses.
 - **Elastic IP (EIP) Support:** With NLB, users can assign Elastic IPs for easier IP management, making it simpler to integrate into existing network configurations.
 - **Connection-Based Targeting:** NLB is connection-oriented, distributing traffic to targets based on established connections rather than individual requests.
 - **TLS Termination:** NLB can terminate TLS at the load balancer level, offloading decryption processes from the backend servers.
 - **NLB Target Group Protocols:** TCP, UDP, TCP_UDP, and TLS.

NLB Use Cases

- Use cases: Network Load Balancers are ideal for latency-sensitive applications, real-time applications, and applications requiring raw TCP/UDP load balancing without complex routing logic. It's commonly used for gaming, VoIP services, and IoT applications.



NLB Limitation

- **No Cookie-Based Sticky Sessions:** An ALB can track user sessions using HTTP cookies. An NLB cannot read cookies; its version of "stickiness" relies strictly on the client's IP address, which can cause uneven traffic distribution if many users share a single gateway IP (like a corporate office network).
- **No Path or Host Routing:** You cannot route traffic based on URL structures (like sending /api to one group of servers and /images to another) or host headers (like app.domain.com vs web.domain.com).

Target Group ALB & NLB

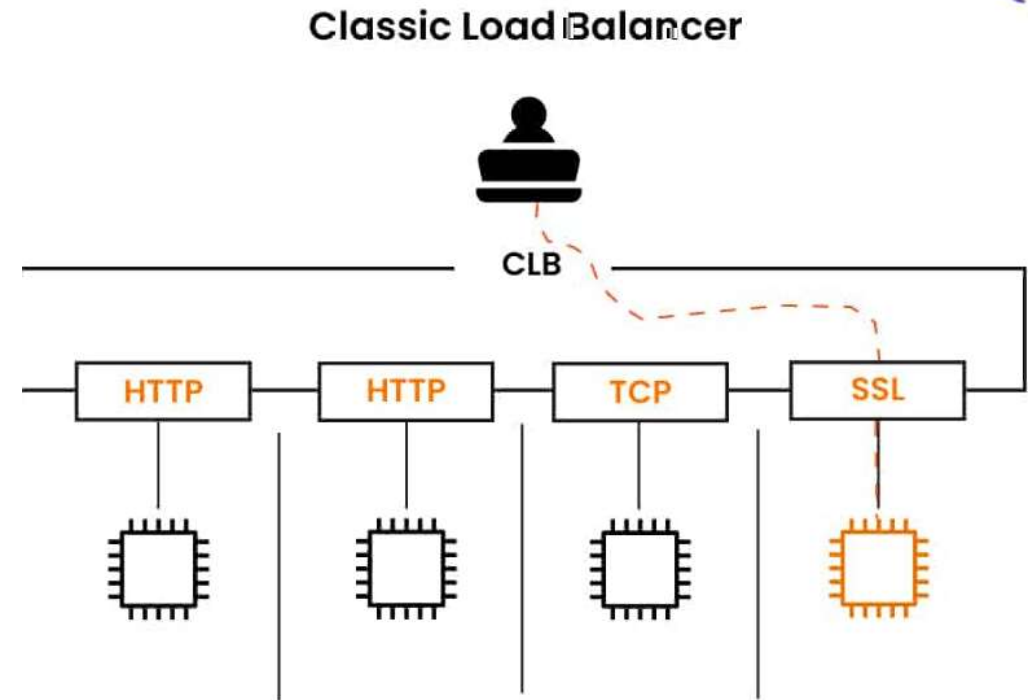
Target Type	Supported by ALB?	Supported by NLB?	Description
Instance	Yes	Yes	Route traffic to EC2 instances by their Instance ID.
IP	Yes	Yes	Route traffic to specific IP addresses (including pods in EKS or on-premises IPs via Direct Connect).
Lambda	Yes	No	Triggers serverless AWS Lambda functions.
Application Load Balancer	No	Yes	Routes Layer 4 traffic from the NLB directly into an ALB (useful for combining a static IP with Layer 7

ALB VS NLB

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)
OSI Layer	Layer 7 (Application)	Layer 4 (Transport)
Traffic Type	HTTP, HTTPS, HTTP/2, gRPC	TCP, UDP, TLS
Performance	Optimized for complex routing; slight latency overhead.	Ultra-low latency (sub-millisecond) and scales to millions of requests per second instantly.
IP Addresses	Uses dynamic IP addresses (changes over time).	Provides Static IP addresses per Availability Zone (supports Elastic IPs).
Routing Capability	Advanced rules based on path, host, headers, query strings, and cookies.	Strict routing based on IP protocol, source/destination IP, and ports.
	EC2, Containers	EC2, Containers, Fargate

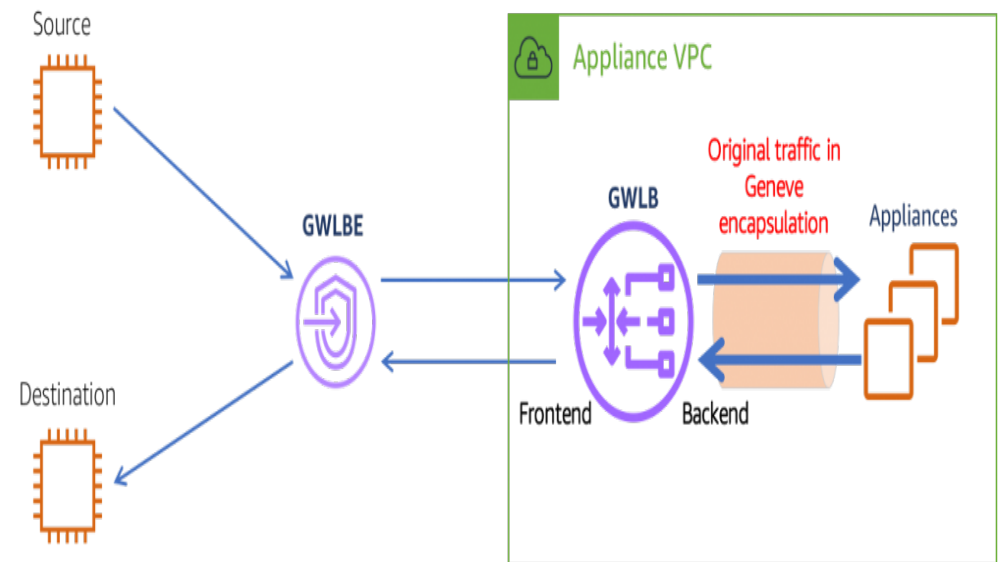
Classic Load Balancer

- Supports TCP (Layer 4), HTTP & HTTPS (Layer 7)
- Health checks are TCP or HTTP based
- Fixed hostname
 - `XXX.region.elb.amazonaws.com`



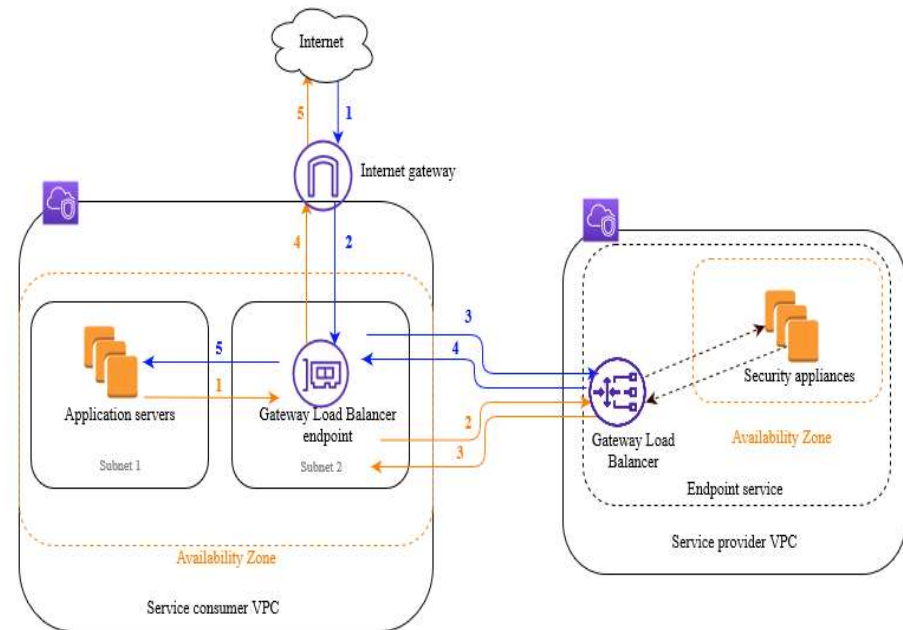
Gateway Load Balancer

- An AWS **Gateway Load Balancer (GWLB)** is a completely different compared to an ALB or NLB.
- GWLB is a **transparent "bump-in-the-wire" network gateway**.
- It is designed to route all incoming and outgoing network traffic through appliances—such as **firewalls, Intrusion Detection/Prevention Systems (IDS/IPS), or deep packet inspection tools**—before that traffic reaches its final destination.



How GWLB works?

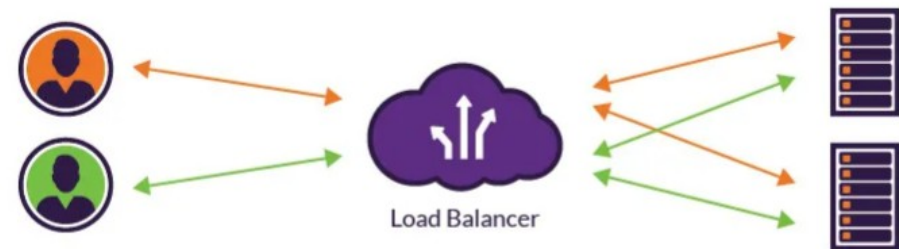
- Gateway Load Balancer operates at **Layer 3 (Network Layer)**.
- Unlike other load balancers, the client and the application server have no idea the GWLB is even there.
- Traffic entering your network (e.g., from the Internet Gateway) is routed first to a **Gateway Load Balancer Endpoint (GWLBE)**
- GWLB takes the original, unmodified client packet and wraps (encapsulates) it inside a **GENEVE protocol** network packet (on UDP port 6081).
- It forwards this encapsulated packet to one of your healthy third-party firewall appliances. The firewall unwraps it, inspects it for threats, re-encapsulates it, and sends it back to the GWLB.
- GWLB strips away the GENEVE wrapper and forwards the pristine, original IP packet to your actual application servers.



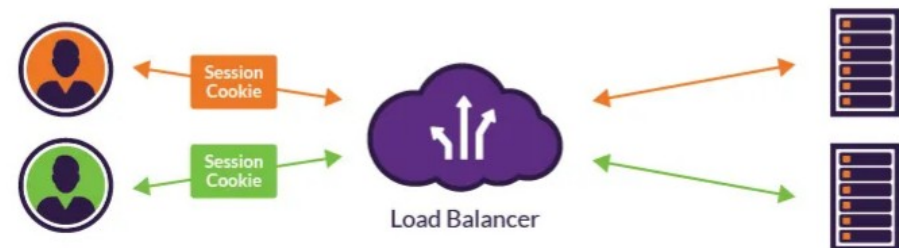
Sticky Session

- Normally, a load balancer distributes requests evenly across a pool of servers using algorithms like Round Robin.
- A **Sticky Session** (also known as **Session Affinity**) is a mechanism used by load balancers to route a user's repeated requests to the **exact same backend server** for the entire duration of their session.
- Load balancers achieve stickiness primarily using **HTTP Cookies** (at Layer 7).
- ALB Supports **Cookie-Based Stickiness** (both duration-based and application-based). Since it reads Layer 7 traffic, it handles HTTP cookies natively.
- **NLB** Does *not* understand HTTP cookies. Instead, it supports **Source IP Stickiness**. It binds a client to a server based strictly on the client's IP address. (Note: If the client changes networks—like switching from Wi-Fi to cellular—their IP changes, and their stickiness breaks).

Without Session Stickiness



With Session Stickiness



Load balancers mainly use round-robin algorithms for most traffic.

How Sticky Sessions Work (The Mechanism)

Load balancers achieve stickiness primarily using **HTTP Cookies** (at Layer 7). There are two main ways this is configured:

A. Duration-Based (Load Balancer-Generated) Stickiness

- The load balancer itself generates a special cookie (e.g., AWSALB in AWS) and inserts it into the HTTP response header when the user first visits.
- The cookie contains information pointing to the specific backend server.
- The cookie has a preset expiration time (Time-To-Live). As long as the cookie is valid, the load balancer reads it on every incoming request and routes it to the designated server.

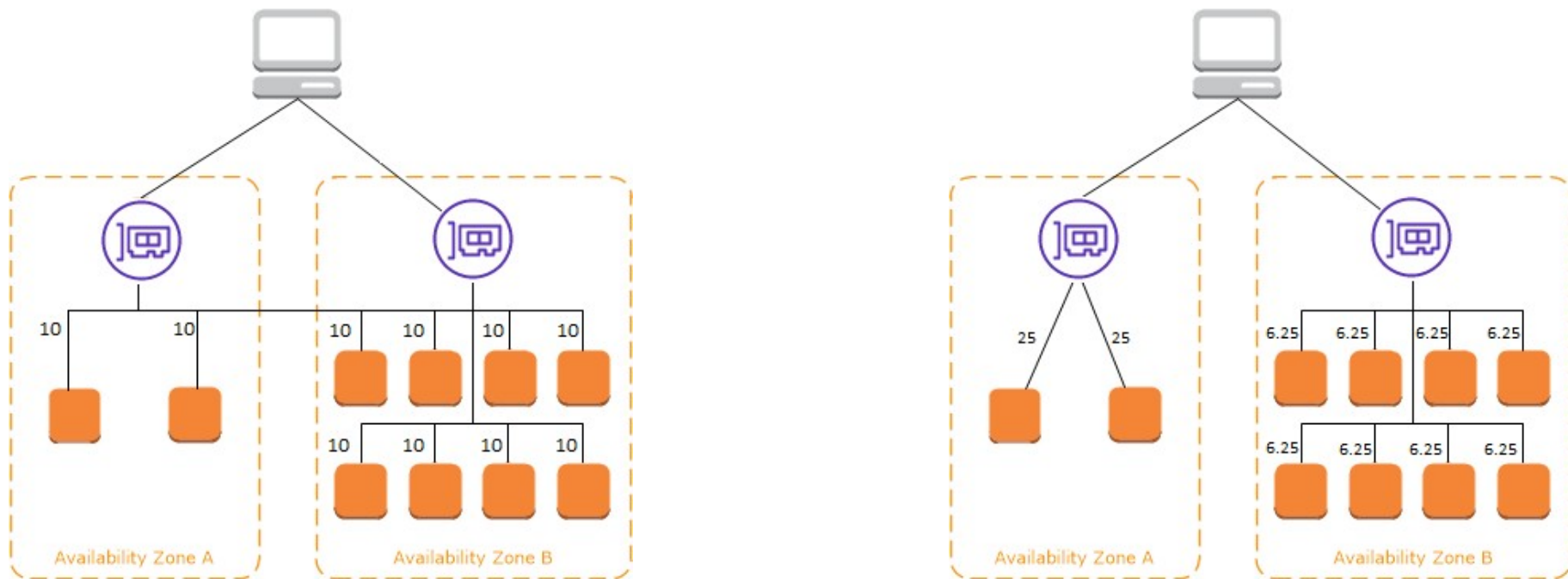
B. Application-Controlled Stickiness

- Your backend web application generates its own session cookie (e.g., JSESSIONID or PHPSESSID) to track the user.
- The load balancer intercepts this cookie and learns which server generated it.
- The load balancer then creates a shadow cookie or maps that application cookie to ensure future requests stay tied to that same backend server.

Cross-Zone Load Balancing

- Imagine you have **10 backend EC2 instances** spread unevenly across two zones:
 - **AZ-A:** Has **2** instances.
 - **AZ-B:** Has **8** instances.
- Cross-Zone Load Balancing is DISABLED
 - **AZ-A's node** gets 50% of the total traffic. It divides it equally among its 2 instances → **Each instance gets 25% of the total traffic.**
 - **AZ-B's node** gets 50% of the total traffic. It divides it equally among its 8 instances → **Each instance gets only 6.25% of the total traffic.**
- Cross-Zone Load Balancing is ENABLED
 - Traffic hits the nodes (50/50 split).
 - The nodes seamlessly distribute the traffic across all 10 instances regardless of where they live.
 - **Result: Every single instance receives exactly 10% of the total traffic.**

Cross Zone Load balancing



Note: Network Load Balancer & Gateway Load Balancer Disabled by default & on ALB it is enable by default.

Deregistration Delay or Connection draining

- Core Problem: Imagine a user is downloading a large 50 MB PDF file from your website, which is being served by **EC2 InstanceA**. Suddenly, your AWS Auto Scaling group decides to scale down because traffic is low, or you push a new software deployment that marks **InstanceA** to be terminated.
- **Without Connection Draining:** The load balancer immediately cuts off all traffic to **InstanceA**. The user's download instantly fails halfway through, and they get a broken network connection error.
- **With Connection Draining:** The load balancer handles the server removal gracefully without disrupting the user.

Auto Scaling Groups (ASG)

- Before Auto Scaling, if a website got a sudden spike in traffic.
- Servers would crash due to high load.
- If traffic was low ,companies would waste money keeping idle servers running.
- An ASG solves both problems:
 - **High Availability (Self-Healing).**
 - **Cost Efficiency.**
 - **Smooth Scalability.**

How ASG work?

1. **The Blueprint (The Launch Template):** Before an ASG can boot up a single server, it needs to know *what* to build. It gets this information from a **Launch Template**. Template Contains:
 - The Operating System (AMI)
 - Instance Type (e.g., t3.medium)
 - Security Groups and IAM Roles
 - User Data (the startup scripts that clone your app code and boot up services)
2. **The Monitoring Phase (Health Checks):** Once the instances are running, the ASG acts as a strict supervisor. It continuously checks if your servers are alive using two types of health checks:
 - **EC2 Status Checks (Default):** The ASG monitors the underlying AWS hardware and hypervisor. If the physical rack freezes or loses power, the ASG marks the instance as unhealthy.
 - **ELB/Target Group Health Checks (Recommended):** If your ASG is linked to an Application Load Balancer, the ASG can listen to the balancer's application pings. If your web server code crashes (e.g., returns HTTP 500 errors) but the server stays powered on, the ASG will recognize it as broken.

How ASG work? (Continue..)

3. The Self-Healing Loop (Replacing Bad Servers): If an instance fails its health checks, the ASG automatically triggers a **Replace** workflow:

- **Terminate:** The ASG stops routing new traffic to the broken instance and terminates it.
- **Launch:** It instantly looks at your Launch Template and provisions a brand-new instance in its place.
- **Maintain boundaries:** This ensures your pool of running servers never drops below your configured **Minimum Size** or **Desired Capacity**.

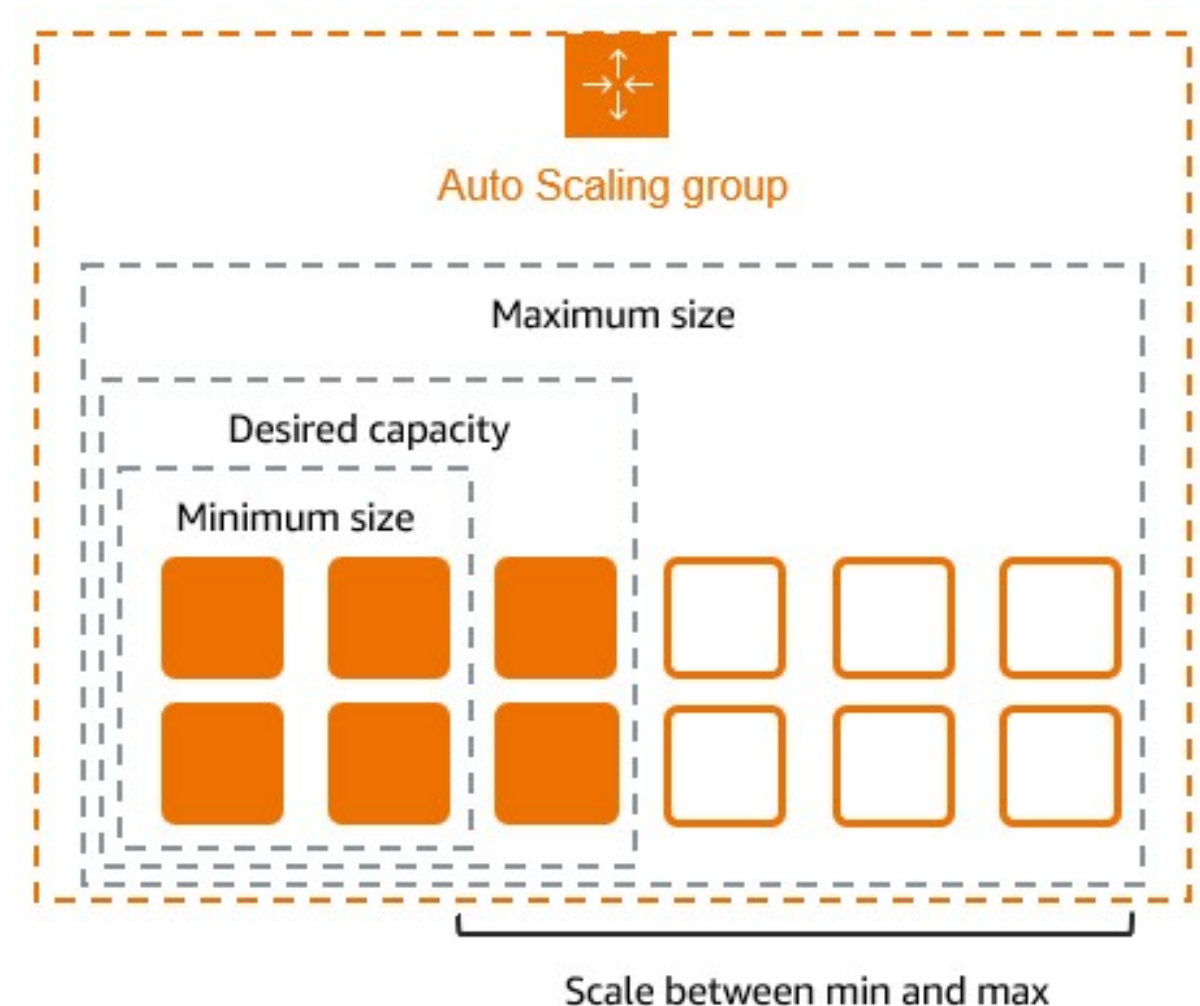
4. The Scaling Loop (Responding to Traffic): When traffic changes, the ASG adjusts your fleet size using Amazon **CloudWatch Alarms**.

- Scaling Out (Adding Capacity)
- Scaling In (Removing Capacity)

5. The Cooldown Period (Preventing Panic): To prevent the ASG from oscillating wildly (e.g., scaling up 2 servers, then immediately scaling down 1 server a minute later), AWS enforces a **Cooldown Period** (typically 300 seconds by default).

Key Configurations of an ASG

- When you set up an ASG, you define three main size parameters that control its boundaries:
 - **Minimum Size.**
 - **Maximum Size.**
 - **Desired Capacity.**



Scaling Policies in ASG

1. Target Tracking Scaling (The Set-and-Forget Default)

- This is the most popular and recommended policy for standard applications. Instead of configuring complex logic, you simply **choose a metric and define a target number** you want to maintain.
- **Example:** ASG automatically calculates how many instances to add if the CPU spikes above 60%, and exactly how many to terminate if the CPU drops below 60%.

2. Dynamic Scaling (Step and Simple Scaling)

- Dynamic scaling scales your fleet based on explicit thresholds you define using CloudWatch alarms.

A. Step Scaling (Highly Recommended over Simple)

- **Example:** * If CPU is between 50% and 70% → **Add 1 instance.**
 - If CPU is between 70% and 85% → **Add 3 instances** (Step 2).

B. Simple Scaling (Legacy)

- Example : Simple scaling only allows a single response to an alarm (e.g., "*If CPU > 70%, add 1 instance*"). Once triggered, it completely locks down and waits for a cool-down period before it can evaluate traffic again. It is largely superseded by Step Scaling.

Scaling Policies in ASG

3. Scheduled Scaling (Predictable Events)

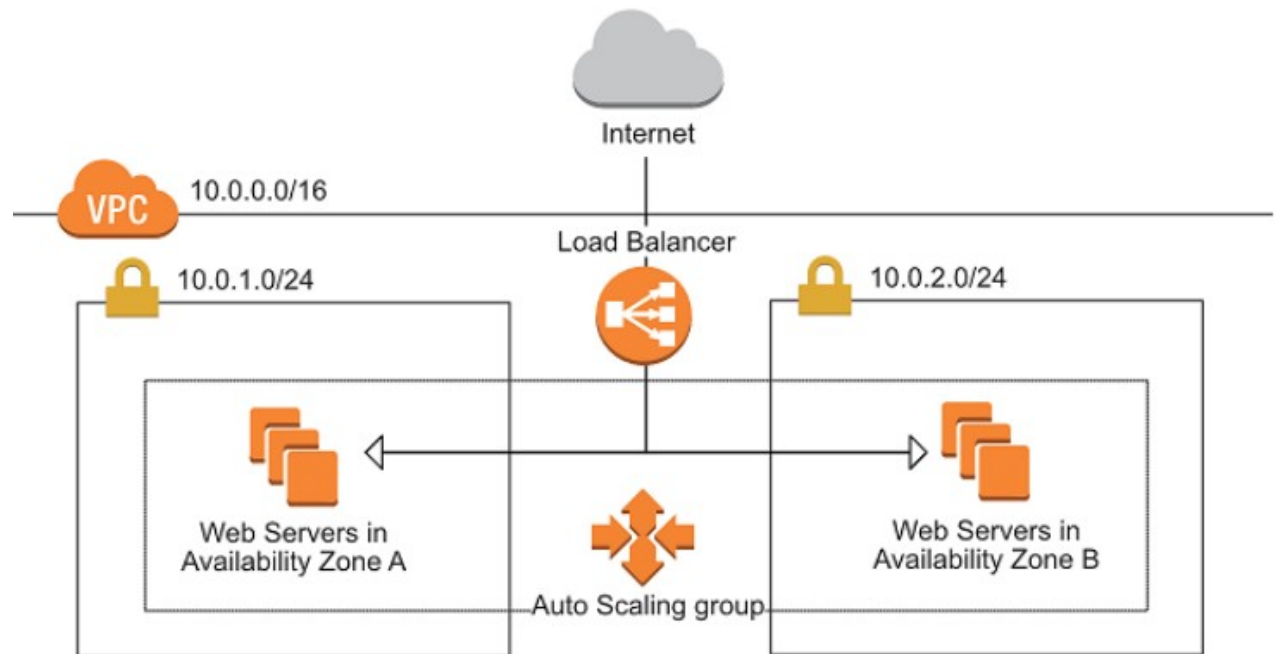
- Scheduled scaling ignores live metrics entirely. Instead, it scales your infrastructure based on **time and date**.
- **Example:** An e-commerce platform scheduling a scale-out to 20 instances 15 minutes *before* a major midnight flash sale begins.

4. Predictive Scaling (AI-Driven)

- Predictive scaling uses machine learning to analyze your historical traffic patterns (up to 14 days of data) and **forecast your upcoming load**.
- **Example :** It builds a 24-hour predictive traffic model. If it predicts that you will experience a massive traffic surge tomorrow at 2:00 PM, it will proactively spin up instances at 1:30 PM so they are fully booted, warm, and ready *before* the traffic actually lands.

ASG with AWS load balancer

- When you pair an **Auto Scaling Group (ASG)** with an **Elastic Load Balancer (ELB/ALB)**, you are implementing highly available, self-healing, and scalable web application in AWS.



SSL/TLS - Basics

- An SSL/TLS certificate enables **in-transit encryption**, securing the data pipeline between your clients and the load balancer.
- While **TLS (Transport Layer Security)** is the modern, updated protocol version, it is still commonly referred to as **SSL (Secure Sockets Layer)**.
- Public certificates are issued and verified by trusted **Certificate Authorities (CAs)**, such as Let's Encrypt, DigiCert, and GoDaddy.
- Every certificate includes a specific **expiration date** and must be periodically renewed to maintain a secure connection.

AWS Load Balancer – SSL Certificate

- Think of a load balancer with an SSL certificate as a **secure security gate** at the entrance of a business park.
- Load balancer uses a standard digital security file called an **X.509 certificate** to lock and protect your web traffic.
- You can easily get and manage these files for free using **ACM (AWS Certificate Manager)**, or you can choose to upload your own custom certificate from another company.
- When a user visits your site, their browser uses a feature called **SNI (Server Name Indication)** to tell the load balancer exactly which website name they are trying to open (e.g., api.com vs app.com). The load balancer then hands them the correct matching certificate.

Problem SNI Solves

- If you have app.com and api.com sharing the *same* load balancer IP address, the load balancer has to present an SSL certificate to the browser immediately.
- But since the browser hasn't sent the HTTP data yet, the load balancer has no idea which website the user is trying to visit.
- With SNI enabled, the browser includes the target hostname (e.g., api.com) inside that initial unencrypted greeting.
- The ALB reads this SNI hostname *and* pulls the exact SSL certificate for api.com and sends it back to the browser.
- Note: **Application Load Balancer (ALB) and Network Load Balancer (NLB) Fully Support the SNI.**

